

Snowball: Extracting Relations from Large Plain-Text Collections

Eugene Agichtein Luis Gravano
Department of Computer Science
Columbia University
12 14 Amsterdam Avenue
New York, NY 10027-7003, USA
{eugene,gravano}@cs.columbia.edu

ABSTRACT

Text documents often contain valuable structured data that is hidden in regular English sentences. This data is best exploited if available as a relational table that we could use for answering precise queries or for running data mining tasks. We explore a technique for extracting such tables from document collections that requires only a handful of training examples from users. These examples are used to generate extraction patterns, that in turn result in new tuples being extracted from the document collection. We build on this idea and present our *Snowball* system. *Snowball* introduces novel strategies for generating patterns and extracting tuples from plain-text documents. At each iteration of the extraction process, *Snowball* evaluates the quality of these patterns and tuples without human intervention, and keeps only the most reliable ones for the next iteration. In this paper we also develop a scalable evaluation methodology and metrics for our task, and present a thorough experimental evaluation of *Snowball* and comparable techniques over a collection of more than 300,000 newspaper documents.

1 INTRODUCTION

Text documents often hide valuable *structured data*. For example, a collection of newspaper articles might contain information on the *location* of the headquarters of a number of *organizations*. If we need to find the location of the headquarters of, say, Microsoft, we could try and use traditional information-retrieval techniques for finding documents that contain the answer to our query [13]. Alternatively, we could answer such a query more precisely if we somehow had available a *table* listing all the organization-location pairs that are mentioned in our document collection. A *tuple* $\langle o, \ell \rangle$ in such table would indicate that the headquarters of organization o are in location ℓ , and that this information was present in a document in our collection. Tuple $\langle \text{Microsoft}, \text{Redmond} \rangle$ in our table would then provide the answer to our

query. The web contains millions of pages whose text hides data that would be best exploited in structured form. In this paper we develop the *Snowball* system for extracting structured data from plain-text documents with *minimal human participation*. Our techniques build on the idea of DIPRE introduced by Brin [3].

DIPRE: Dual Iterative Pattern Expansion DIPRE was proposed as an approach for extracting a structured *relation* (or *table*) from a collection of HTML documents. The method works best in an environment like the World-Wide Web, where the table *tuples* to be extracted will tend to appear in uniform contexts repeatedly in the collection documents (i.e., in the available HTML pages). DIPRE exploits this redundancy and inherent structure in the collection to extract the target relation with minimal training from a user.

As in the rest of the paper, we focus the presentation on the organization-location scenario defined above. In this context DIPRE's goal is to extract a table with all the organization-location tuples that appear in a given document collection. Initially, we provide DIPRE with a handful of instances of valid organization-location pairs. For example, we may indicate that $\langle \text{Microsoft}, \text{Redmond} \rangle$ is a valid pair, meaning that Microsoft is an organization whose headquarters are located in Redmond. Similarly, we provide DIPRE with a few other examples, as Table 1 shows. In addition, the user provides a general regular expression that the entities must match. This is all the training that DIPRE requires from the user.

Organization	Location of Headquarters
MICROSOFT	REDMOND
EXXON	IRVING
IBM	ARMONK
BOEING	SEATTLE
INTEL	SANTA CLARA

Table 1: User-provided example tuples for DIPRE.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

Digital Libraries, San Antonio, TX.

Copyright 2000 ACM 1-58 113-23 | -X/00/0006.. \$5.00

After this initial training phase, DIPRE looks for instances of the example organizations and locations in the text documents. Then, DIPRE examines the text that surrounds the initial tuples. For example, DIPRE inspects the context surrounding Microsoft and Redmond in “*computer servers at Microsoft’s headquarters in Redmond*” to construct a pat-

tern “<STRING1>’s headquarters in <STRING2>.” Other possible patterns are listed in Figure 1.

A DIPRE pattern consists of a five tuple <order, urlprefix, left, middle, right> and is generated by grouping together occurrences of seed tuples that have equal strings separating the entities (middle) and then setting the left and right strings to the longest common substrings of the context on the left and on the right of the entities, respectively. The order reflects the order in which the entities appear, and urlprefix is set to the longest common substring of the source URL’s where the seed tuples were discovered. After generating a number of patterns from the initial seed tuples, DIPRE scans the available documents in search of segments of text that match the patterns. As a result of this process, DIPRE generates new tuples and uses them as the new “seed.” DIPRE starts the process all over again by searching for these new tuples in the documents to identify new promising patterns.

<STRING1>’s headquarters in <STRING2>
 <STRING2>-based <STRING1>
 <STRING1>, <STRING2>

Figure 1: Initial DIPRE patterns. <STRING1> and <STRING2> are regular expressions that would match an organization and a location, respectively.

Related Work Brin’s DIPRE method and our *Snowball* system that we introduce in this paper both address issues that have long been the subject of information extraction research. Our task, though, is different in that we do not attempt to extract *all* the relevant information from each document, which has been the goal of traditional information extraction systems [10]. One of the major challenges in information extraction is the necessary amount of manual labor involved in training the system for each new task. This challenge has been addressed in different ways. One approach is to build a powerful and intuitive graphical user interface for training the system, so that domain experts can quickly adopt the system for each new task [14]. Nevertheless, these systems still require substantial expert manual labor to port the system to each new domain. In contrast, *Snowball* and DIPRE require only a handful of example tuples for each new scenario.

Another approach is to train the system over a large *manually tagged* corpus, where the system can apply machine learning techniques to generate extraction patterns [8]. The difficulty with this approach is the need for a large tagged corpus, which again involves a significant amount of manual labor to create. To combat this problem, some methods have been proposed to use an untagged corpus for training. [11] describes generating extraction patterns automatically by using a training corpus of documents that were manually marked as either relevant or irrelevant for the topic. This approach requires less manual labor than to tag the documents, but nevertheless the effort involved is substantial. [6] describes machine learning techniques for creating a knowl-

edge base from the web, consisting of classes of entities and relations, by exploiting the content of the documents, as well as the link structure of the web. This method requires training over a large set of web pages, with relevant document segments manually labeled, as well as a large training set of page-to-page relations.

Finally, a number of systems use unlabeled examples for training. This direction of research is closest to our work. Specifically, the approach we are following falls into the broad category of bootstrapping techniques. Bootstrapping has been an attractive alternative in automatic text processing. [15] demonstrates a bootstrapping technique for disambiguating senses of ambiguous nouns. [5] uses bootstrapping to classify named entities in text exploiting two orthogonal features, i.e., the spelling of the entity itself (e.g., having a suffix “Corp.”), and the context in which the entity occurs. [12] also presents a bootstrapping technique to extract patterns to recognize and classify named entities in text. [16] describes an extension of DIPRE to mining the Web for acronyms and their expansions. [2] presents a methodology and theoretical framework for combining unlabeled examples with labeled examples to boost performance of a learning algorithm for classifying web pages. While the underlying principle of using the systems’ output to generate the training input for the next iteration is the same for all of these approaches, the tasks are different enough to require specialized methodologies.

Our Contributions As we have discussed, [3] describes a method for extracting relations from the web using bootstrapping. Our *Snowball* system, which we present in this paper, builds on this work. Our main contributions include:

- **Techniques for generating patterns and extracting tuples:** We develop a new strategy for defining and representing patterns that is at the same time flexible, so that we capture most of the tuples that are hidden in the text in our collection, and selective, so that we do not generate invalid tuples (Sections 2.1 and 2.2).
- **Strategies for evaluating patterns and tuples:** Since the amount of training that *Snowball* requires is minimal, it is crucial that the patterns and tuples that are generated during the extraction process be evaluated. This way, *Snowball* will be able to eliminate unreliable tuples and patterns from further consideration. We develop strategies for estimating the reliability of the extracted patterns and tuples (Section 2.3).
- **Evaluation methodology and metrics:** Evaluating systems like *Snowball* and DIPRE is challenging: these systems are designed to work over large document collections, so manually inspecting all documents to build the “perfect” table that should be extracted is just not feasible. We introduce a scalable evaluation methodology and associated metrics (Section 3), which we use in Sections 4 and 5 for large-scale experiments over collections of training and test documents. These collections have a total of over 300,000 real documents.

2 THE SNOWBALL SYSTEM

In this section we present the *Snowball* system (Figure 2), which develops key components of the basic DIPRE method. More specifically, *Snowball* presents a novel technique to generate patterns and extract tuples from text documents (Sections 2.1 and 2.2). Also, *Snowball* introduces a strategy for evaluating the quality of the patterns and the tuples that are generated in each iteration of the extraction process (Section 2.3). Only those tuples and patterns that are regarded as being “sufficiently reliable” will be kept by *Snowball* for the following iterations of the system (Section 2.3). These new strategies for generation and filtering of patterns and tuples improve the quality of the extracted tables significantly, as the experimental evaluation in Section 5 will show.

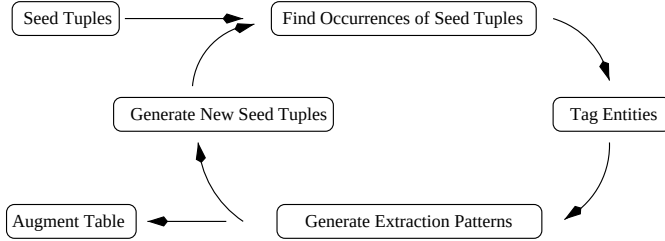


Figure 2: The main components of *Snowball*.

2.1 Generating Patterns

A crucial step in the table extraction process is the generation of patterns to find new tuples in the documents. Ideally, we would like patterns both to be *selective*, so that they do not generate incorrect tuples, and to have high *coverage*, so that they identify many new tuples. In this section, we introduce a novel way of generating such patterns from a set of seed tuples and a document collection.

Snowball is initially given a handful of example tuples. For every such organization-location tuple $\langle o, \ell \rangle$, *Snowball* finds segments of text in the document collection where o and ℓ occur close to each other, just as DIPRE does, and analyzes the text that “connects” o and ℓ to generate patterns. A key improvement of *Snowball* from the basic DIPRE method is that *Snowball*’s patterns include named-entity tags. An example of such a pattern is $\langle \text{LOCATION} \rangle\text{-based} \langle \text{ORGANIZATION} \rangle$. This pattern will not match any pair of strings connected by “-based.” Instead, $\langle \text{LOCATION} \rangle$ will only match a string identified by a tagger as an entity of type *LOCATION*. Similarly, $\langle \text{ORGANIZATION} \rangle$ will only match a string identified by a tagger as an entity of type *ORGANIZATION*. Figure 3 shows additional patterns that *Snowball* might generate, with named-entity tags.

$\langle \text{ORGANIZATION} \rangle$ ’s headquarters in $\langle \text{LOCATION} \rangle$
 $\langle \text{LOCATION} \rangle$ -based $\langle \text{ORGANIZATION} \rangle$
 $\langle \text{ORGANIZATION} \rangle$, $\langle \text{LOCATION} \rangle$

Figure 3: Patterns that exploit named-entity tags.

A key step in generating and later matching patterns like the one above is finding where $\langle \text{ORGANIZATION} \rangle$ and $\langle \text{LOCATION} \rangle$ entities occur in the text. For this, *Snowball* uses a state-of-the-art named-entity tagger, The MITRE Corporation’s Alembic Workbench [7]. In addition to *ORGANIZATION* and *LOCATION* entities, Alembic can identify *PERSON* entities, and can be trained to recognize other kinds of entities. (See Section 6 for further discussion.) Once the entities in the text documents are tagged, *Snowball* can ignore unwanted entities (e.g., *PERSON*s), focus on occurrences of *LOCATION* and *ORGANIZATION* entities, and analyze the context that surrounds each pair of such entities to check if they are connected by the right words and hence match our patterns.

Snowball represents the context around the *ORGANIZATION* and *LOCATION* entities in the patterns in a flexible way that produces patterns that are selective, yet have high coverage. As a result, minor variations such as an extra comma or a determiner will not stop us from matching contexts that are otherwise close to our patterns. More specifically, *Snowball* represents the left, middle, and right “contexts” associated with a pattern just like the vector-space model of information retrieval represents documents and queries [13]. Thus, the *left*, *middle*, and *right* contexts are three vectors associating weights (i.e., numbers between 0 and 1) with terms (i.e., arbitrary strings of non-space characters). These weights indicate the importance of each term in the corresponding context.

Definition 1 A *Snowball* pattern is a 5-tuple $\langle \text{left}, \text{tag1}, \text{middle}, \text{tag2}, \text{right} \rangle$, where *tag1* and *tag2* are named-entity tags, and *left*, *middle*, and *right* are vectors associating weights with terms.

An example of a *Snowball* pattern is a 5-tuple $\langle \{ \langle \text{the}, 0.2 \rangle \}, \text{LOCATION}, \{ \langle -, 0.5 \rangle, \langle \text{based}, 0.5 \rangle \}, \text{ORGANIZATION}, \{ \} \rangle$. This pattern will match strings like “the Irving-based Exxon Corporation,” where the word “the” (left context) precedes a location (Irving), which is in turn followed by the strings “-” and “based” (middle context) and an organization. Slight variations of the given string will also match the pattern to a smaller extent. (We introduce a notion of “degree of match” later in this section.)

To match text portions with our 5-tuple representation of patterns, *Snowball* also associates an equivalent 5-tuple with each document portion that contains two named entities with the correct tag (i.e., *LOCATION* and *ORGANIZATION* in our scenario). After identifying two such entities in a string S , *Snowball* creates three weight vectors l_S , r_S , and m_S from S by analyzing the left, right, and middle contexts around the named entities, respectively. Each vector has a non-zero weight for each term that appears in the respective context where the l_S and r_S are each limited to the w -term window to the left and to the right of the entity pair. The weight of

a term in each vector is a function of the frequency of the term in the corresponding context. These vectors are scaled so their norm is one. Finally, they are multiplied by a scaling factor to indicate each vector's relative importance. From our experiments with English-language documents, we have found the middle context to be the most indicative of the relationship between the elements of the tuple. Hence we will typically assign the terms in the middle vector higher weights than the left and right vectors. After extracting the 5-tuple representation of string S , *Snowball* matches it against the 5-tuple pattern by taking the inner product of the corresponding left, middle, and right vectors.

Definition 2 The degree of match $Match(t_P, t_S)$ between two 5-tuples $t_P = \langle l_P, t_1, m_P, t_2, r_P \rangle$ (with tags t_1 and t_2) and $t_S = \langle l_S, t'_1, m_S, t'_2, r_S \rangle$ (with tags t'_1 and t'_2) is defined as:

$$Match(t_P, t_S) = \begin{cases} l_P \cdot l_S + m_P \cdot m_S + r_P \cdot r_S & \text{if the tags match} \\ 0 & \text{otherwise} \end{cases}$$

In order to generate a pattern, *Snowball* groups occurrences of known tuples in documents, if the contexts surrounding the tuples are “similar enough.” More precisely, *Snowball* generates a 5-tuple for each string where a seed tuple occurs, and then clusters these 5-tuples using a simple single-pass clustering algorithm [9], using the *Match* function defined above to compute the similarity between the vectors and some minimum similarity threshold τ_{sim} . The left vectors in the 5-tuples of clusters are represented by a centroid $\bar{l}_s$. Similarly, we collapse the middle and right vectors into $\bar{m}_s$ and $\bar{r}_s$, respectively. These three centroids, together with the original tags (which are the same for all the 5-tuples in the cluster), form a *Snowball* pattern $\langle \bar{l}_s, t_1, \bar{m}_s, t_2, \bar{r}_s \rangle$.

2.2 Generating Tuples

After generating patterns (Section 2.1), *Snowball* scans the collection to discover new tuples. The basic algorithm is outlined in Figure 4.

Snowball first identifies sentences that include an organization and a location, as determined by the named-entity tagger. For a given text segment, with an associated organization o and location ℓ , *Snowball* generates the 5-tuple $t = \langle l_c, t_1, m_c, t_2, r_c \rangle$. A candidate tuple $\langle o, \ell \rangle$ is generated if there is a pattern t_p such that $Match(t, t_p) \geq \tau_{sim}$, where τ_{sim} is the clustering similarity threshold of Section 2.1.

Each candidate tuple will then have a number of patterns that helped generate it, each with an associated degree of match. *Snowball* uses this information, together with information about the selectivity of the patterns, to decide what candidate tuples to actually add to the table that it is constructing.

2.3 Evaluating Patterns and Tuples

Generating good patterns is challenging. For example, we may generate a pattern $\langle \{\}, ORGANIZATION, \langle “,” \rangle, 1 \rangle$, LO-

```
sub GenerateTuples(Patterns)
  foreach text_segment in corpus
    (1)  $\langle o, \ell \rangle, \langle l_s, t_1, m_s, t_2, r_s \rangle =$ 
       = CreateOccurrence(text_segment);
        $T_C = \langle o, \ell \rangle$ ;
        $Sim_{Best} = 0$ ;
       foreach  $p$  in Patterns
    (2)  $sim = Match(\langle l_s, t_1, m_s, t_2, r_s \rangle, p)$ ;
       if ( $sim \geq \tau_{sim}$ )
    (3) UpdatePatternSelectivity( $p, T_C$ );
       if ( $sim \geq Sim_{Best}$ )
          $Sim_{Best} = sim$ ;
          $P_{Best} = p$ ;
       if ( $Sim_{Best} \geq \tau_{sim}$ )
         CandidateTuples[ $T_C$ ].Patterns[ $P_{Best}$ ] =
           =  $Sim_{Best}$ ;
  return CandidateTuples;
```

Figure 4: Algorithm for extracting new tuples using a set of patterns.

CATION, $\{\}$ from text occurrences like “Intel, Santa Clara, announced...” This pattern will be matched by any string that includes an organization followed by a comma, followed by a location. Estimating the *confidence* of the patterns, so that we do not trust patterns that tend to generate wrong tuples, is one of the problems that we address in this section. We can weigh the *Snowball* patterns based on their selectivity, and trust the tuples that they generate accordingly. Thus, a pattern that is not selective will have a low weight. The tuples generated by such a pattern will be discarded, unless they are supported by selective patterns.

The case for tuples is analogous. “Bad” seed tuples may generate extraneous patterns that in turn might result in even more wrong tuples in the next *Snowball* iteration. To prevent this, we only keep tuples with high *confidence*. The confidence of the tuple is a function of the selectivity and the number of the patterns that generated it. Intuitively, the confidence of a tuple will be high if it is generated by several highly selective patterns.

The pattern and tuple evaluation is the key part of our system, and is responsible for most of the improvement over the DIPRE scheme. As an initial filter, we eliminate all patterns supported by fewer than τ_{sup} seed tuples. We then update the *confidence* of each pattern in Step (3) of the algorithm in Figure 4, which checks each candidate tuple $t = \langle o, \ell \rangle$ generated by the pattern in question. If there is a high confidence tuple $t' = \langle o, \ell' \rangle$ generated during an earlier iteration of the system for the same organization o as in t , then this function compares locations ℓ and ℓ' . If the two locations are the same, then the tuple t is considered a *positive* match for the pattern. Otherwise, the match is *negative*. Intuitively, the candidate tuple that a pattern generates for the “known” organizations should match the locations of these organizations. Otherwise, the confidence in this pattern will be low. Note that this confidence computation assumes that organi-

ization is a key for the relation that we are extracting (i.e., two different tuples in a valid instance of the relation cannot agree on the organization attribute). Estimating the confidence of the *Snowball* patterns for relations without such a single-attribute key is part of our future work (Section 6).

Definition 3 The confidence of a pattern P is:

$$Conf(P) = \frac{P.positive}{(P.positive + P.negative)}$$

where $P.positive$ is the number of positive matches for P and $P.negative$ is the number of negative matches.

As an example, consider the pattern $P = \langle \{\}, ORGANIZATION, <“, 1>, LOCATION, \{\} \rangle$ referred to above. Assume that this pattern only matches the three lines of text below:

“Exxon, Irving, said”

“Intel, Santa Clara, cut prices”

“invest in Microsoft, New York-based analyst Jane Smith said”

The first two lines generate candidate tuples $\langle Exxon, Irving \rangle$ and $\langle Intel, Santa Clara \rangle$, which we already knew from previous iterations of the system. The third line generates tuple $\langle Microsoft, New York \rangle$. The location in this tuple conflicts with the location in tuple $\langle Microsoft, Redmond \rangle$, hence this last line is considered a negative example. Then, pattern P has confidence $Conf(P) = \frac{2}{2+1} = 0.67$.

Our definition of confidence of a pattern above is only one among many possibilities. An alternative is to account for a pattern’s coverage in addition to its selectivity. For this, we adopt a metric originally proposed by Riloff [11] to evaluate extraction patterns generated by the Autoslog-TS information extraction system, and define $Conf_{RlogF}(P)$ of pattern P as follows.

Definition 4 The RlogF confidence of pattern P is:

$$Conf_{RlogF}(P) = Conf(P) \cdot \log_2(P.positive)$$

Pattern confidences are defined to have values between 0 and 1. Therefore, we normalize the $Conf_{RlogF}$ values by dividing them by the largest confidence value of any pattern.

Having scored the patterns, we are now able to evaluate the new candidate tuples. Recall that for each tuple we store the set of patterns that produced it, together with the measure of similarity between the context in which the tuple occurred, and the matching pattern. Consider a candidate tuple T and the set of patterns $P = \{P_i\}$ that were used to generate T . Let us assume for the moment that we know the probability $Prob(P_i)$ with which each pattern P_i generates valid tuples. If these probabilities are independent of each other, then the probability that T is valid, $Prob(T)$, can be calculated as:

$$Prob(T) = 1 - \prod_{i=0}^{|P|} (1 - Prob(P_i))$$

Our confidence metric $Conf(P_i)$ was designed to be a rough estimate of $Prob(P_i)$, the probability of pattern P_i generating a valid tuple. We also account for the cases where T has occurred in contexts that did not match our patterns perfectly. Intuitively, the lower the degree of match between a pattern and a context, the higher is the chance of producing an invalid tuple. For this, we scale each $Conf(P_i)$ term by the degree of match of the corresponding pattern and context:

Definition 5 The confidence of a candidate tuple T is:

$$Conf(T) = 1 - \prod_{i=0}^{|P|} (1 - (Conf(P_i) \cdot Match(C_i, P_i)))$$

where $P = \{P_i\}$ is the set of patterns that generated T and C_i is the context associated with an occurrence of T that matched P_i with degree of match $Match(C_i, P_i)$.

Note that when we described the calculation of the pattern confidence, we ignored any confidence values from previous iterations of *Snowball*. To control the learning rate of the system, we set the new confidence of the pattern as:

$$Conf(P) = Conf_{new}(P) \cdot W_{updt} + Conf_{old}(P) \cdot (1 - W_{updt})$$

If parameter $W_{updt} < 0.5$ then the system in effect trusts new examples less on each iteration, which will lead to more conservative patterns and have a damping effect. For our experiments we set $W_{updt} = 0.5$. We also adjust the confidence of already-seen tuples in an analogous way.

After determining the confidence of the candidate tuples using the definition above, *Snowball* discards all tuples with low confidence. These tuples could add noise into the pattern generation process, which would in turn introduce more invalid tuples, degrading the performance of the system. The set of tuples to use as the seed in the next *Snowball* iteration is then $Seed = \{T | Conf(T) > \tau_t\}$, where τ_t is some prespecified threshold.

For illustration purposes, Table 2 lists three representative patterns that *Snowball* extracted from the document collection that we describe in Section 4.1.

Conf	middle	right
1	$\langle \text{based}, 0.53 \rangle$ $\langle \text{in}, 0.53 \rangle$	$\langle , , 0.01 \rangle$
0.69	$\langle ', 0.42 \rangle$ $\langle \text{s}, 0.42 \rangle$ $\langle \text{headquarters}, 0.42 \rangle$ $\langle \text{in}, 0.12 \rangle$	
0.61	$\langle (, 0.93 \rangle$	$\langle \rangle, 0.12 \rangle$

Table 2: Actual patterns discovered by *Snowball*. (For each pattern the left vector is empty, tag1 = ORGANIZATION, and tag2 = LOCATION.)

3 EVALUATION METHODOLOGY AND METRICS

The goal of *Snowball* is to extract as many valid tuples as possible from the text collection and to combine them into one table. As we have discussed, we do not attempt to capture every *instance* of such tuples. Instead, we exploit the fact that these tuples will tend to appear multiple times in the types of collections that we consider. As long as we capture one instance of such a tuple, we will consider our system to be successful for that tuple. This is different from the goal of traditional information extraction [1]. Traditional information extraction systems aim at extracting all the relevant information from *each document* as completely as possible, while our system extracts tuples from all of the documents in the collection and combines them into one table. To evaluate this task, we adapt the recall and precision metrics from information retrieval to quantify how accurate and comprehensive our *combined table of tuples* is. Our metric for evaluating the performance of an extraction system over a collection of documents D is based on determining *Ideal*, the set of all the tuples that appear in the collection D (Section 3.1). After identifying *Ideal*, we compare it against the tuples produced by the system, *Extracted*, using the adapted precision and recall metrics (Section 3.2).

3.1 Methodology for Creating the *Ideal* Set

For small text collections, we could inspect all documents manually and compile the *Ideal* table by hand. Unfortunately, this evaluation approach does not scale, and becomes infeasible for the kind of large collections over which *Snowball* is designed to operate. To address this problem, we start by considering a large, publicly available directory of 13,000 organizations provided on the “Hoover’s Online” web site¹. From this well structured directory, we generate a table of organization-location pairs. Unfortunately, we cannot use this table as is, since some of the organizations in it might not occur at all in our collection.

To determine the target set of tuples *Ideal* from the Hoover’s-compiled table above, we need to keep only the tuples that have the organization mentioned together with their location in a document. To find all such instances, we identify all the variations of each organization name in the Hoover’s table as they may appear in the collection, and then check if the headquarters of the test organization are mentioned nearby. We used Whirl [4], a research tool developed at AT&T Research Laboratories for integrating similar textual information, to match each organization name, as it occurs in the collection, to the organization in the Hoover’s table.

3.2 The *Ideal* Metric

Now that we have created the *Ideal* table, we can use it to evaluate the quality of the *Snowball* output, the *Extracted* table. If the initial directory of organizations from Hoover’s contained all possible organizations, then we could just measure what fraction of the tuples in *Extracted* are in *Ideal* (pre-

cision) and what fraction of the tuples in *Ideal* are in *Extracted* (recall). Unfortunately, a large collection will contain many more tuples that are contained in any single manually compiled directory. (In our estimate, our training collection contains more than 80,000 valid organization-location tuples.) If we just calculated precision as above, all the valid tuples extracted by *Snowball*, which are not contained in our *Ideal* set, will unfairly lower the reported value of precision for the system.

To address this problem we create a new table, *Join*, as the join of tables *Ideal* and *Extracted* on a unique key (i.e., organization). For each tuple $T = \langle o, \ell \rangle$ in the *Ideal* table, we find a matching tuple $T' = \langle o', \ell' \rangle$ in the *Extracted* table (if any), such that $o \simeq o'$. (We describe how to deal with variations in the organization names in Section 3.3.) Using these values, we now create a new tuple $\langle o, \ell, \ell' \rangle$ and include it in the *Join* table.

Given the table *Ideal* and the *Join* table that we have just created, we can define recall and precision more formally. We define *Recall* as:

$$Recall = \frac{\sum_{i=0}^{|Join|} [\ell_i = \ell'_i]}{|Ideal|} \cdot 100\% \quad (1)$$

where $[\ell_i = \ell'_i]$ is equal to 1 if the test value ℓ_i matches the extracted value ℓ'_i , and 0 otherwise. Thus, the sum in the numerator is the number of *correct* tuples of the *Ideal* set that we extracted, which we divide by the size of the *Ideal* table to obtain our recall. Similarly, we define *Precision* as:

$$Precision = \frac{\sum_{i=0}^{|Join|} [\ell_i = \ell'_i]}{|Join|} \cdot 100\% \quad (2)$$

An alternative to using our *Ideal* metric to estimate precision could be to sample the extracted table, and check each value in the sample tuples by hand. (Similarly, we could estimate the recall of the system by sampling documents in the collection, and checking how many of the tuples mentioned in those documents the system discovers.) By sampling the extracted table we can detect invalid tuples whose organization is not mentioned in the Hoover’s directory that we used to determine *Ideal*, for example. Similarly, we can detect invalid tuples that result from named-entity tagging errors. Hence, we also report precision estimates using sampling in Section 5.

3.3 Matching Location and Organization Names

A problem with calculating the *Ideal* metric above is introduced by the proliferation of variants of organization names. We combine all variations into one, by using a *self-join* of the *Extracted* table with itself. We use Whirl to match the organization names to each other, to create the table *Extracted'*. We pick an arbitrary variation of the organization name, o_s , as the “standard,” and pick a location, ℓ_{max} , from the set of

¹<http://www.hoovers.com>

matching organization-location tuples, with the highest confidence value. We then insert the tuple $\langle o_s, \ell_{max} \rangle$ into the *Extracted*² table.

Similarly, we need to decide when the location extracted for an organization is correct. For example, our system might conclude that California is the location of the headquarters of Intel. This answer is correct, although not as specific as could be. Our scoring system will in fact consider a tuple $\langle Intel, California \rangle$ as correct. Specifically, we consider tuple $\langle o, \ell \rangle$ to be valid if (a) organization o is based in the U.S. and ℓ is the city or state where o 's headquarters are based; or (b) organization o is based outside of the U.S. and ℓ is the city or country where o 's headquarters are based.

4 EXPERIMENTAL SETTING

We describe the training and text collections that we used for experiments in Section 4.1. We also enumerate the different extraction methods that we compare experimentally (Section 4.2).

4.1 Training and Test Collections

Our experiments use large collections of real newspapers from the North American News Text Corpus, available from LDC². This corpus includes articles from the Los Angeles Times, The Wall Street Journal, and The New York Times for 1994 to 1997. We split the corpus into two collections: training and test. The *training* collection consists of 178,000 documents, all from 1996. The *test* collection is composed of 142,000 documents, from 1995 and 1997.

Both *Snowball* and DIPRE rely on tuples appearing multiple times in the document collection at hand. To analyze how “redundant” the training and test collections are, we report in Table 3 the number of tuples in the *Ideal* set for each frequency level. For example, 5455 organizations in the *Ideal* set are mentioned in the training collection, and 3787 of them are mentioned in the same line of text with their location at least once. So, if we wanted to evaluate how our system performs on extracting tuples that occur at least once in the training collection, the *Ideal* set that we will create for this evaluation will contain 3787 tuples.

Occurrences:	Organization-Location Pairs	
	Training Collection	Test Collection
0	5455	4642
1	3787	3411
2	2774	2184
5	1321	909
10	593	389

Table 3: Occurrence statistics of the test tuples in the experiment collections.

The first row of Table 3, corresponding to zero occurrences, deserves further explanation. If we wanted to evaluate the

performance of our system on *all* the organizations that were mentioned in the corpus, even if the appropriate location never occurred near its organization name anywhere in the collection, we would include all these organizations in our *Ideal* set. So, if the system attempts to “guess” the value of the location for such an organization, any value that the system extracts will automatically be considered wrong in our evaluation.

4.2 Evaluating Alternative Techniques

We compared *Snowball* with two other techniques, the *Baseline* method and our implementation of the DIPRE method. These two methods require minimal or no training input from the user, and hence are comparable with *Snowball* in this respect. In contrast, state-of-the-art information extraction systems require substantial manual labor to train the system, or to create a hand-tagged training corpus.

The first method, *Baseline*, is based purely on the frequency of co-occurrence of the organization and the location. Specifically, *Baseline* reports the location that co-occurs in the same line with each organization most often as the headquarters for this organization.

The second method is DIPRE. We did not have access to its original implementation, so we had to re-implement it and adapt it to our collections. The original DIPRE implementation uses *urlprefix* to restrict pattern generation and application. Since all of our documents came from just three sources, DIPRE was not able to exploit this feature. The second, more important modification had to do with the fact that DIPRE was designed to extract tuples from HTML-marked data. Without HTML tags, DIPRE could not find occurrences of the seed tuples in plain text that were surrounded by exactly the same non-empty contexts. To solve this problem, we used the named-entity tagger to pre-tag the input to DIPRE. This way, all the organizations and locations were consistently surrounded by named-entity tags. DIPRE could then generate patterns that take advantage of these tags. The results we report are *not* for the original DIPRE implementation, but rather for our adaptation for tagged documents.

4.3 Snowball

We explored the best parameter values for *Snowball* by running the system on the training corpus. Parameters we experimented with include:

- **Use of Punctuation:** We experimented with discarding punctuation and other non-alphanumeric characters from the contexts surrounding the entities. Our hypothesis was that punctuation may just add noise and carry little content to help extract tuples. We report results for *Snowball* and *Snowball-Plain*, where *Snowball* uses punctuation, and *Snowball-Plain* discards it.
- **Choice of Pattern Scoring Strategies:** We tried variations on the basic framework for weighing patterns, as described in Section 2, with or without using the *RlogF* metric of [11].

²<http://www.ldc.upenn.edu>

Parameter	Value	Description
τ_{sim}	0.6	minimum degree of match (Section 2.1)
τ_t	0.8	minimum tuple confidence (Section 2.3)
τ_{sup}	2	minimum pattern support (Section 2.1)
I_{max}	3	number of iterations of <i>Snowball</i>
W_{middle}	0.6	weight for the <i>middle</i> context (Section 2.1)
W_{left}	0.2	weight for the <i>left</i> context (Section 2.1)
W_{right}	0.2	weight for the <i>right</i> context (Section 2.1)

Table 4: Parameter values used for evaluating *Snowball* on the test collection.

- **Choice of Pattern Similarity Threshold (τ_{sim}):** This parameter controls how flexible the patterns are, both during the pattern generation stage (i.e., how similar the occurrences of the example tuples have to be in order to be grouped into one cluster), as well as during the tuple extraction stage, where τ_{sim} controls the minimum similarity between the context surrounding the potential tuple and a pattern, determining whether a tuple will be generated.
- **Choice of Tuple Confidence Threshold (τ_t):** This threshold determines the minimum confidence a tuple must have to be included in the seed set to start the next iteration.

5 EXPERIMENTAL RESULTS

In this section, we experimentally compare the performance of *Snowball* and the alternative techniques that we discussed in Section 4.2. Our experiments use the training and test collections of Section 4.1. We ran experiments on the training collection to determine the optimal pattern scoring strategy, optimal values for τ_{sim} , τ_t , τ_{sup} , and the optimal weight distribution W_{left} , W_{middle} , and W_{right} for the left, middle, and right context vectors of each pattern.

As we discussed, the only input to the *Snowball* system during the evaluation on the test collection were the five seed tuples of Table 1. All the extraction patterns were learned from scratch by running the *Snowball* system using the operational parameters listed in Table 4, which worked best on the training collection. The normalized *RlogF* metric was used to score patterns for generating the set of seed tuples for the next iteration. The results are reported in Figure 5. The plot shows the performance of the systems as we attempt to extract test tuples that are mentioned more times in the corpus. As we can see, *Snowball* performs increasingly well as the number of times that the test tuples are required to be mentioned in the collection is increased. While DIPRE has better precision than *Snowball* at the 0-occurrence level (72% vs. 67% for *Snowball*), *Snowball* has at all occurrence levels significantly higher recall than DIPRE and *Baseline* do. We also observe that punctuation matters. The recall of *Snowball-Plain* is significantly lower than that of *Snowball*.

Figure 6 shows that *Snowball*’s results are stable over subsequent iterations of the algorithm. In contrast, DIPRE quickly diverges, since it has no way to prevent unreliable tuples from being seed for its next iteration. We report data for only two

iterations for *Snowball-Plain* because it converged after the second iteration (i.e., it did not produce any new seed tuples).

As discussed in Section 3.2, we complete our evaluation of the precision of the extraction systems by manually examining a sample of their output. For this, we randomly selected 100 tuples from each of the extracted tables, and checked whether each of these tuples was a valid organization-location pair or not. We separate the errors into three categories: errors due to mistagging a location and assigning it to a valid organization (“Location” error), errors due to including a non-existing organization (“Organization” error), and errors due to deducing an incorrect relationship between a valid organization and location (“Relationship” error). These different types of errors are significant because they highlight different “culprits”: the “Location” and “Organization” errors could be prevented if we had a perfect named-entity tagger, whereas the “Relationship” errors are wholly the extraction system’s fault (Table 5).

The last column in Table 5 (P_{Ideal}) is precision, calculated by ignoring the “Organization” errors and computing the fraction of valid organizations for which a correct location was found. These values correspond to the values of precision we would have calculated if our *Ideal* table included all the valid organizations in the random samples. These figures, however, do not capture invalid tuples generated due to improper tagging of a string as an organization. From our manual inspection of a random sample of 100 tuples from each extracted table, we observed that DIPRE’s sample contained 74 correct tuples and 26 incorrect ones. *Snowball*’s sample contained 52 correct tuples and 48 incorrect tuples, while *Baseline* has a majority of incorrect tuples (25 vs. 75). As we can see from the breakup of the errors in the table, virtually all of *Snowball*’s errors are tagging related (i.e., “Location” or “Organization” errors). If we prune the *Snowball*’s final output to only include those tuples t with $Conf(t) \geq 0.8 = \tau_t$, then most of these spurious tuples disappear. In effect, from a random sample of 100 tuples from this pruned table, 93 tuples are valid and only 7 are invalid. Furthermore, none of the invalid tuples are due to “Relationship” errors (third row of Table 5).

So far, the results that we have reported for *Snowball* are based on a table that contains all the “candidate” tuples generated during *Snowball*’s last iteration. As we saw in Table 5, the precision of *Snowball*’s answer varies dramatically if we prune this table using the tuple confidence threshold τ_t . Of course, this last-step pruning is likely to result in lower recall values. In Figure 7 we explore the tradeoff between precision and recall for different values of this last-step pruning threshold. A user who is interested in high-precision tables might want to use high values for this threshold, while a user who is interested in high-recall tables might want to use lower values of the threshold. For example, by setting $\tau_t = 0.4$ and filtering the *Extracted* table accordingly, we estimate the absolute precision of *Snowball*’s output to be 76% and recall

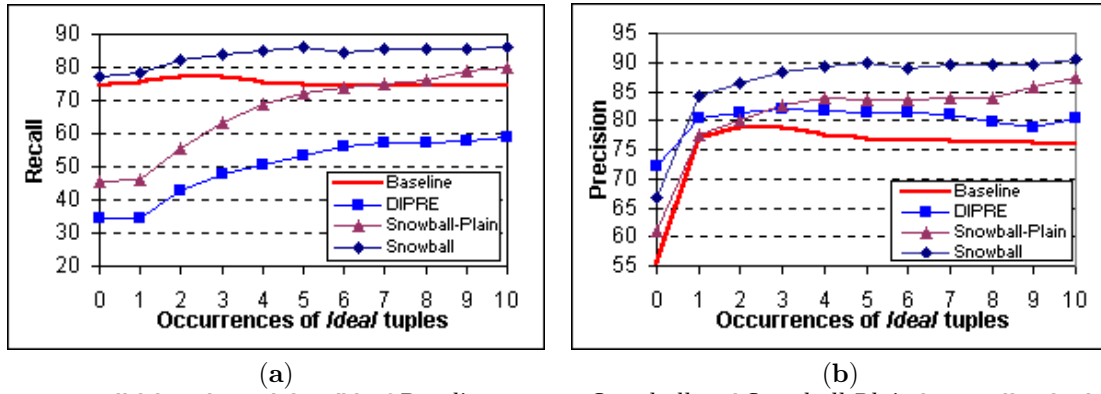


Figure 5: Recall (a) and precision (b) of *Baseline*, *DIPRE*, *Snowball* and *Snowball-Plain* (test collection).

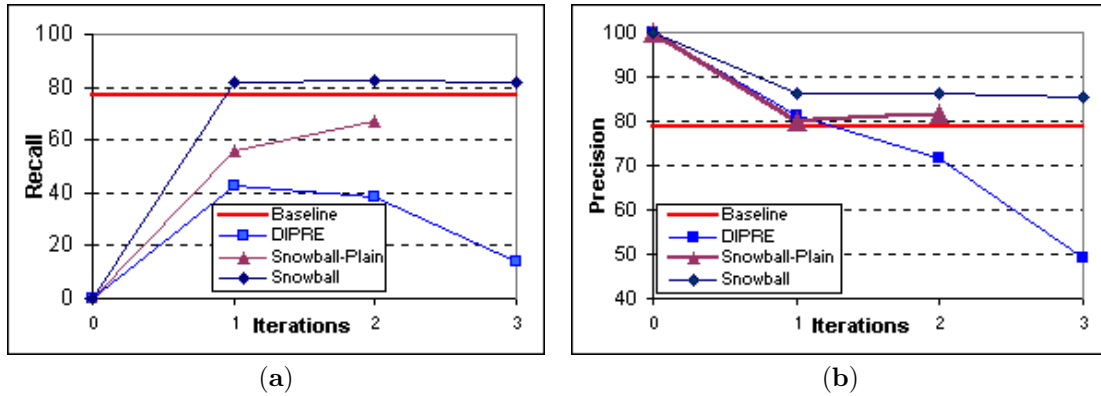


Figure 6: Recall (a) and precision (b) of *Baseline*, *DIPRE*, *Snowball*, and *Snowball-Plain* as a function of the number of iterations (*Ideal tuples with occurrence ≥ 2* ; test collection).

			Type of Error			P_{Ideal}
	Correct	Incorrect	Location	Organization	Relationship	
DIPRE	74	26	3	18	5	90%
<i>Snowball</i> (all tuples)	52	48	6	41	1	88%
<i>Snowball</i> ($\tau_t = 0.8$)	93	7	3	4	0	96%
<i>Baseline</i>	25	75	8	62	5	66%

Table 5: Manually computed precision estimate, derived from a random sample of 100 tuples from each extracted table.

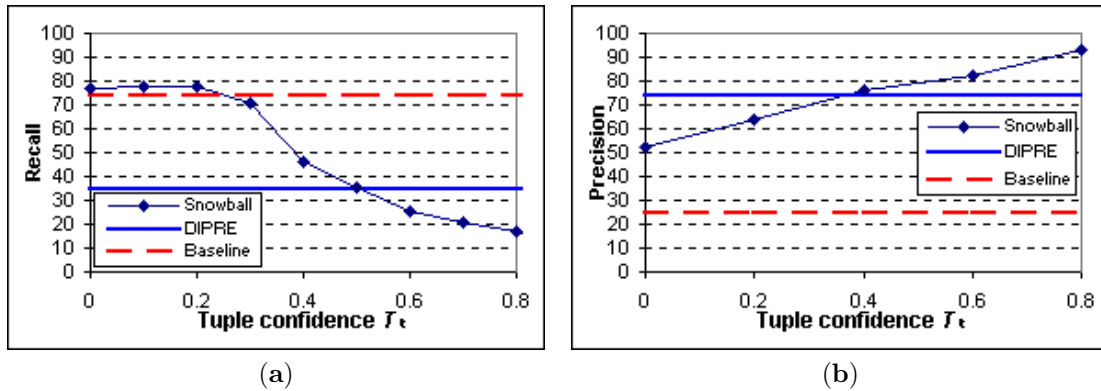


Figure 7: Recall (a) and sample-based precision (b) as a function of the threshold τ_t used for the last-step pruning of the *Snowball* tables (*Ideal tuples with occurrence ≥ 1* ; test collection).

to be 45%, both of which are higher than the corresponding metrics of DIPRE's output.

In summary, both *Snowball* and DIPRE show significantly higher precision than *Baseline*. In effect, *Baseline* tends to generate many tuples, which results in high recall at the expense of low precision. *Snowball*'s recall is at least as high as that of *Baseline* for most of the tests, with higher precision values. *Snowball*'s recall is generally higher than DIPRE's, while the precision of both techniques is comparable.

6 CONCLUSIONS AND FUTURE WORK

This paper presents *Snowball*, a system for extracting relations from large collections of plain-text documents that requires minimal training for each new scenario. We introduced novel strategies for generating extraction patterns for *Snowball*, as well as techniques for evaluating the quality of the patterns and tuples generated at each step of the extraction process. Our large-scale experimental evaluation of our system shows that the new techniques produce high-quality tables, according to the scalable evaluation methodology that we introduce in this paper. Our experiments involved over 300,000 newspaper articles.

We only evaluated our techniques on plain text documents, and it would require future work to adopt our methodology to HTML data. While HTML tags can be naturally incorporated into *Snowball*'s pattern representation, it is problematic to extract named-entity tags from arbitrary HTML documents. State-of-the-art taggers rely on clues from the text surrounding each entity, which may be absent in HTML documents that often rely on visual formatting to convey information. On a related note, we have assumed throughout that the attributes of the relation we extract (i.e., organization and location) correspond to named entities that our tagger can identify accurately. As we mentioned, named-entity taggers like Alembic can be extended to recognize entities that are distinct in a context-independent way (e.g., numbers, dates, proper names). For some other attributes, we will need to extend *Snowball* so that its pattern generation and matching could be anchored around, say, a noun phrase as opposed to a named entity as in this paper. In the future, we will also generalize *Snowball* to relations of more than two attributes. Finally, a crucial open problem is how to generalize our tuple and pattern evaluation strategy of Section 2.3 so that it does not rely on an attribute being a key for the relation.

ACKNOWLEDGMENTS

This material is based upon work supported by the National Science Foundation under Grant No. IIS-9733880. We also thank Regina Barzilay, Ralph Grishman, and Vasilis Hatzivassiloglou for their helpful comments, and Eleazar Eskin for many fruitful discussions.

REFERENCES

1. *Proceedings of the Sixth Message Understanding Conference*. Morgan Kaufman, 1995.
2. Avrim Blum and Tom Mitchell. Combining labeled and unlabeled data with co-training. In *Proceedings of the 1998 Conference on Computational Learning Theory*, 1998.
3. Sergey Brin. Extracting patterns and relations from the World-Wide Web. In *Proceedings of the 1998 International Workshop on the Web and Databases (WebDB'98)*, March 1998.
4. William Cohen. Integration of heterogeneous databases without common domains using queries based on textual similarity. In *Proceedings of the 1998 ACM International Conference on Management of Data (SIGMOD'98)*, 1998.
5. Michael Collins and Yoram Singer. Unsupervised models for named entity classification. In *Proceedings of the Joint SIG-DAT Conference on Empirical Methods in Natural Language Processing and Very Large Corpora*, 1999.
6. M. Craven, D. DiPasquo, D. Freitag, A. McCallum, T. Mitchell, K. Nigam, and S. Slattery. Learning to construct knowledge bases from the World Wide Web. *Artificial Intelligence*, 1999.
7. David Day, John Aberdeen, Lynette Hirschman, Robyn Kozierok, Patricia Robinson, and Marc Vilain. Mixed-initiative development of language processing systems. In *Proceedings of the Fifth ACL Conference on Applied Natural Language Processing*, April 1997.
8. D. Fisher, S. Soderland, J. McCarthy, F. Feng, and W. Lehnert. Description of the UMass systems as used for MUC-6. In *Proceedings of the 6th Message Understanding Conference*. Columbia, MD, 1995.
9. William B. Frakes and Ricardo Baeza-Yates, editors. *Information Retrieval: Data Structures and Algorithms*. Prentice-Hall, 1992.
10. Ralph Grishman. Information extraction: Techniques and challenges. In *Information Extraction (International Summer School SCIE-97)*. Springer-Verlag, 1997.
11. Ellen Riloff. Automatically generating extraction patterns from untagged text. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, pages 1044–1049, 1996.
12. Ellen Riloff and Rosie Jones. Learning dictionaries for information extraction by multi-level bootstrapping. In *Proceedings of the Sixteenth National Conference on Artificial Intelligence*, 1999.
13. Gerard Salton. *Automatic Text Processing: The transformation, analysis, and retrieval of information by computer*. Addison-Wesley, 1989.
14. Roman Yangarber and Ralph Grishman. NYU: Description of the Proteus/PET system as used for MUC-7. In *Proceedings of the Seventh Message Understanding Conference (MUC-7)*. Morgan Kaufman, 1998.
15. D. Yarowsky. Unsupervised word sense disambiguation rivaling supervised methods. In *Proceedings of the 33rd Annual Meeting of the Association for Computational Linguistics*, pages 189–196. Cambridge, MA, 1995.
16. Jeonghee Yi and Neel Sundaresan. Mining the web for acronyms using the duality of patterns and relations. In *Proceedings of the 1999 Workshop on Web Information and Data Management*, 1999.